

Base de datos — Cómo se usa PostgreSQL en el proyecto

Este documento explica **todo** lo relacionado con la base de datos del Sistema de Prácticas Preprofesionales: cómo se conecta Python con PostgreSQL, cómo se crea el esquema, qué consultas se ejecutan para guardar y mostrar datos, cómo se generan los identificadores, cómo se manejan las transacciones y las contraseñas. Está pensado para poder **exponer el proyecto** sabiendo exactamente qué hace.

Todo el acceso a la base de datos está concentrado en un solo archivo:

`persistencia/gestor_persistencia.py` (clase `GestorPersistencia`). La configuración de la conexión está en `persistencia/config_bd.py`. El esquema como documentación está en `persistencia/esquema_postgresql.sql`. El cifrado de contraseñas en `modelo/seguridad.py`.

Diagramas (entidad-relación en notación de Chen + diagrama relacional): ver `diagramas_bd.md`. El archivo `diagrama_er_chen.dot` genera el E-R como imagen con Graphviz.

1. Tecnología y decisiones de diseño

- **Motor:** PostgreSQL. **Driver de Python:** `psycopg2` (en `requirements.txt`).
- **Esquema lógico:** todas las tablas viven en el esquema `practicas` (no en `public`).
- **Identificadores:** claves naturales donde corresponde (usuario del admin, identificador del login, cédula de las personas) e **identificadores subrogados generados por la base** con `GENERATED ALWAYS AS IDENTITY` (oferta, postulación, práctica, solicitud, formularios). La aplicación **no** calcula ids; los recupera con `INSERT ... RETURNING`.
- **Integridad:** claves foráneas, `UNIQUE`, `CHECK`, `NOT NULL`. El borrado es **lógico** (columna `eliminado`), por eso las FK no usan `ON DELETE CASCADE`.
- **Tipos apropiados:** fechas `DATE`, dinero y nota `NUMERIC` (no coma flotante), estructuras anidadas `JSONB`.
- **Seguridad:** las contraseñas se guardan **cifradas** (hash PBKDF2-HMAC-SHA256 con sal); nunca en texto plano.
- **Transacciones:** las operaciones de negocio que tocan varias tablas son atómicas.
- **Arranque:** `python main.py` crea el esquema, tablas, índices y vistas (idempotente) y siembra datos de ejemplo si la base está vacía.

2. Conexión Python ↔ PostgreSQL

2.1 Parámetros (`persistencia/config_bd.py`)

Se leen de **variables de entorno** (no se incrustan credenciales en el código); si no están definidas, se usan valores por defecto de desarrollo:

```
CONFIG_BD = {
    "host": os.environ.get("PGHOST", "localhost"),
    "port": int(os.environ.get("PGPORT", "5432")),
    "dbname": os.environ.get("PGDATABASE", "practicas_db"),
```

```
"user": os.environ.get("PGUSER", "postgres"),
"password": os.environ.get("PGPASSWORD", "postgresql"),
"schema": os.environ.get("PGSCHEMA", "practicas"),
}
```

2.2 Apertura de la conexión

```
self.conexion = psycopg2.connect(
    host=..., port=..., dbname=..., user=..., password=...,
    client_encoding="UTF8") # mensajes de error de PostgreSQL en UTF-8
self._asegurar_esquema() # crea esquema, tablas, índices y vistas
(idempotente)
```

- **Una sola conexión** durante la vida de la app (escritorio mono-usuario).
- Cada consulta usa un cursor: `with self.conexion.cursor() as cur: ....`
- No hay autocommit: se confirma con `commit()` o se revierte con `rollback()`.

2.3 Patrón de ejecución y seguridad ante inyección

```
with self.conexion.cursor() as cur:
    cur.execute("SELECT ... WHERE columna = %s", (valor,))
```

- Los **valores** siempre van como parámetros `%s` (psycopg2 los escapa: evita inyección SQL).
- Los **nombres de tabla/columna** se arman con f-strings a partir del diccionario `MAPEO` (constante del código, no entran del usuario): son seguros.

3. El diccionario MAPEO

Describe toda la estructura. Por entidad: `tabla`, `clase` (a reconstruir, o `None` para dicts como `solicitud`), `clave` (PK), `columnas` = lista de (`nombre`, `tipo_sql`, `restricción`), `checks`, `fks` y `indices`. A partir de él, una sola función genérica construye el DDL y las consultas de cada entidad. Ejemplo de columna con id generado por la base:

```
("id_oferta", "INTEGER", "GENERATED ALWAYS AS IDENTITY PRIMARY KEY"),
```

4. Creación del esquema (DDL)

Al arrancar, `_asegurar_esquema()` recorre `MAPEO` y crea todo de forma idempotente (`CREATE SCHEMA/TABLE/INDEX IF NOT EXISTS`), añadiendo columnas, `CHECK`, claves foráneas e índices, y por último las **vistas**. Ejemplo del SQL generado para `oferta`:

```
CREATE TABLE IF NOT EXISTS "practicas".oferta (
  id_oferta INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  descripcion TEXT NOT NULL,
  puesto VARCHAR(100) NOT NULL,
  fecha_publicacion DATE,
  ruc_empresa VARCHAR(13) NOT NULL,
  eliminado BOOLEAN NOT NULL DEFAULT FALSE,
  CONSTRAINT fk_oferta_ruc_empresa FOREIGN KEY (ruc_empresa)
    REFERENCES "practicas".tutor_empresarial(ruc_empresa)
);
CREATE INDEX IF NOT EXISTS idx_oferta_ruc_empresa ON "practicas".oferta
(ruc_empresa);
```

El orden de creación respeta las dependencias de FK. [persistencia/esquema_postgresql.sql](#) es el espejo fiel de este DDL, para entregarlo como documentación.

5. Integridad y restricciones

Tipo	Dónde	Para qué
PRIMARY KEY natural	administrador.usuario, login.identificador, cédula de personas	Identidad estable provista por el dominio.
PRIMARY KEY subrogada (IDENTITY)	oferta, postulacion, practica, solicitud, formularios	Id entero generado por la base; sin cálculos en la app.
FOREIGN KEY (sin ON DELETE CASCADE)	oferta→tutor_empresarial; postulacion→estudiante/oferta/coordinador; practica→postulacion/tutores; solicitud→estudiante; formularioN→practica	Integridad referencial.
UNIQUE	tutor_empresarial.ruc_empresa, email de personas	Una empresa = un tutor; correos no repetidos.
CHECK	estados, tipo de solicitud, ciclo 1..10 , montos/horas ≥ 0, nota 0..100, rol IN (...)	Valores dentro del conjunto/rango.
NUMERIC	remuneración NUMERIC(10,2) , nota NUMERIC(5,2)	Exactitud en dinero/calificación.
ÍNDICES	columnas FK y de estado más filtradas	Acelerar WHERE/JOIN .

login.identificador no lleva FK (puede apuntar a cualquier tabla de usuario).

6. Operaciones CRUD — las consultas que ejecuta la app

6.1 Obtener por clave — **obtener(entidad, clave)**

```
SELECT cedula, contrasena, ... FROM "practicass".estudiante WHERE cedula = %s
```

6.2 Listar con filtro — `listar(entidad, where, params, orden)`

```
SELECT ... FROM "practicass".postulacion WHERE eliminado = FALSE AND  
estado_validacion = %s
```

Para listas usa `id_oferta = ANY(%s)`.

6.3 Insertar (alta) — `insertar(entidad, objeto)`

Omite la columna IDENTITY y recupera el id generado con `RETURNING`, asignándolo al objeto:

```
INSERT INTO "practicass".oferta (descripcion, puesto, fecha_publicacion,  
ruc_empresa, eliminado)  
VALUES (%s, %s, %s, %s, %s)  
RETURNING id_oferta
```

```
self._asignar(objeto, "id_oferta", cur.fetchone()[0]) # objeto.id_oferta = id  
generado
```

6.4 Actualizar — `actualizar(entidad, objeto)` (UPDATE real)

Modifica la fila existente identificada por su clave primaria (no es un upsert):

```
UPDATE "practicass".postulacion  
SET fecha = %s, estado_validacion = %s, cedula_estudiante = %s, id_oferta = %s,  
id_coordinador = %s, eliminado = %s  
WHERE id_postulacion = %s
```

6.5 Borrado lógico — `marcar_eliminado / marcar_eliminados_por`

```
UPDATE "practicass".estudiante SET eliminado = TRUE WHERE cedula = %s  
UPDATE "practicass".formulario1 SET eliminado = TRUE WHERE id_practica = ANY(%s)
```

6.6 Consulta libre con JOIN — `consultar(sql, params)`

Devuelve filas como dicts; convierte `DATE` a `dd/MM/yyyy`. La usan los listados (sección 8).

Ya **no** existe el antiguo `siguiente_id()` con `MAX(id)+1`: los ids los genera la base.

7. Transacciones (atomicidad por caso de uso)

Cada escritura confirma por sí sola, salvo dentro de `with gestor.transaccion():`, donde todas las escrituras del bloque se confirman juntas (o se revierten con `rollback` ante cualquier error):

```
with self.persistencia.transaccion():
    self.repo_postulaciones.actualizar(postulacion)    # postulación ->
    "Aceptada"
    nueva = self.repo_practicas.agregar(...)           # crea la práctica
```

Se usa en las operaciones que tocan varias tablas: aceptar estudiante + crear práctica, aprobar Formulario 1 + cambiar estado de la práctica, guardar Formulario 2 + cambiar estado, asentar nota + acreditar horas al estudiante, alta de usuario + credencial, y el **borrado en cascada** (lee los hijos y los marca en lote en una sola transacción).

Implementación: un flag interno y un *context manager*:

```
@contextmanager
def transaccion(self):
    anterior = self._en_transaccion
    self._en_transaccion = True
    try:
        yield
        if not anterior: self.conexion.commit()
    except Exception:
        self._rollback_seguro(); raise
    finally:
        self._en_transaccion = anterior
```

Cada método de escritura llama a `_commit_si_corresponde()`, que solo confirma si no hay un bloque `transaccion()` activo.

8. Mostrar datos en la interfaz — vistas con JOIN

Los listados que cruzan tablas se resuelven con **vistas** (`CREATE OR REPLACE VIEW`):

`vista_postulacion_detalle` (postulación + estudiante + oferta + empresa), `vista_practica_detalle` (práctica + estudiante + tutores, con `LEFT JOIN` por tutores opcionales) y `vista_oferta_detalle` (oferta + empresa). El repositorio las consume con `consultar(...)`:

```
self.persistencia.consultar(
    f'SELECT * FROM "{s}".vista_postulacion_detalle '
    f'WHERE estado_validacion = %s AND eliminado = FALSE ORDER BY id_postulacion',
    ("Pendiente",))
```

9. Contraseñas cifradas ([modelo/seguridad.py](#))

- Al guardar cualquier columna `contrasena`, el gestor la cifra con `hash_password()` (PBKDF2-HMAC-SHA256, sal aleatoria, 120 000 iteraciones). Formato almacenado:
`pbkdf2_sha256$<iter>$<sal_hex>$<hash_hex>`.
- El login verifica con `verificar_password(texto_plano, hash)` en tiempo constante.
- En las tablas de la interfaz la contraseña **no se muestra** (se enmascara con `.....`).
- La validación de longitud (4–10) se hace sobre el texto plano antes de cifrar.

10. Reconstrucción de objetos al leer

Se crea el objeto con `Cls.__new__(Cls)` (sin ejecutar `__init__`, para no revalidar datos ya guardados) y se asignan los atributos columna por columna (misma técnica que usan los ORM).

11. Borrado lógico y cascada

Nadie se borra físicamente: `eliminado` pasa a `TRUE` y los listados filtran `WHERE eliminado = FALSE`. La cascada ([controlador/eliminacion_cascada.py](#)) lee los hijos y los marca en lote con `UPDATE ... WHERE columna = ANY(...)`, todo dentro de una transacción.

12. Mapa entidad → tabla

Clase / entidad	Tabla	Clave primaria
Administrador	administrador	usuario (natural)
Credencial	login	identificador (natural)
Estudiante	estudiante	cedula (natural)
TutorAcademico	tutor_academico	cedula (natural)
TutorEmpresarial	tutor_empresarial	cedula (natural)
CoordinadorVinculacion	coordinador_vinculacion	cedula (natural)
Oferta	oferta	id_oferta (IDENTITY)
Postulacion	postulacion	id_postulacion (IDENTITY)
Practica	practica	id_practica (IDENTITY)
Solicitud (dict)	solicitud	id (IDENTITY)
Formulario1/2/3	formulario1/2/3	id_formularioN (IDENTITY)

13. Consultas útiles para verificar/exponer (en [psql](#) o pgAdmin)

```
-- Tablas del esquema
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'practicas' AND table_type = 'BASE TABLE';

-- Claves foráneas y CHECK
SELECT conname, pg_get_constraintdef(oid) FROM pg_constraint
WHERE connamespace = 'practicas'::regnamespace AND contype IN ('f','c');

-- Columnas IDENTITY
SELECT table_name, column_name, is_identity FROM information_schema.columns
WHERE table_schema = 'practicas' AND is_identity = 'YES';

-- Un listado con JOIN ya resuelto (lo que ve la GUI)
SELECT * FROM practicas.vista_postulacion_detalle WHERE eliminado = FALSE;

-- Las contraseñas están cifradas (verás hashes pbkdf2_sha256$, no texto plano)
SELECT identificador, rol, left(contrasena, 18) AS hash_prefijo FROM
practicas.login;

-- La base genera el id (INSERT ... RETURNING)
INSERT INTO practicas.oferta (descripcion, puesto, fecha_publicacion, ruc_empresa)
VALUES ('Demo', 'Pasante', CURRENT_DATE, '0101010106001') RETURNING id_oferta;

-- Un CHECK rechaza datos inválidos
INSERT INTO practicas.estudiante (cedula, contrasena, apellidos, nombres,
telefono,
    email, carrera, ciclo)
VALUES ('1234567890', 'x', 'A', 'B', '0900000000', 'z@z.com', 'C', 99); -- falla:
ciclo > 10
```